Name - Sanjukta Das
CS545 - Machine Learning
Project Report
Performance Evaluation of Various Stock Market Prediction Techniques
Source Code - https://github.com/Sanjukta-Das/CS545-Machine_Learning

---

This project is conducted towards the completion of course work for CS545 - Machine Learning class in Spring-2020. The main purpose behind this project is to evaluate the performance of three different models and find the best model among them to predict the future stock market value with given a dataset of historical values of a specific company (e.g., Intel Corporation). In this project it is demonstrated that, how to tune the model will affect the results, while experimental results show that traditional Machine Learning algorithms may have a better performance.

Introduction:

Anticipating stock market movements consistently is not quite feasible. Predicting stock market prices, have been object of studies for many decades, yet given its inherent intricacies, briskly and chaotic behavior, it has demonstrated to be a very difficult task. Investors and financial specialists perform trading of stocks according to the forecast of stock price patterns. As of late, numerous researchers are focusing on embracing Machine Learning algorithms to anticipate stock value patterns. The number of factors and sources of data and information considered is massive and that makes the task of anticipating stock market prices conduct later on a hard one. For a long time, there have been conversations in Science with respect to the chance of such an accomplishment and it's striking in the related writing that most forecast or prediction models fail to give exact predictions from a general perspective.

Utilizing computationally radical frameworks that utilize machine learning algorithms is progressively common within the advancement of stock trading mechanisms. A few studies, utilizing day by day stock prices, have displayed prescient framework applications trained on fixed periods without considering about new model updates.

The fundamental commitments of this project work are the following :

(1) a prediction model for stock markets using ML algorithms;

(2) the validation of the model using real data from google stock exchange;

(3) evaluation of the model by comparing and analyzing it against some typical baselines.

As each algorithm has its possess advantage and impediment, in this project I studied about 3 different Machine Learning techniques commonly being in utilize for prediction of stock market prices. A comparative approach was taken to discover out ideal strategy for
stock advertise forecast beneath diverse circumstances. Comparison will be made on the premise of

their execution on the same input dataset. At the conclusion, execution of these 3 models is assessed and summarize that which calculation is way better in anticipating stock price.

Selecting Dataset & Pre-Processing:

Collecting good datasets, preprocessing and normalizing data is one of the most vital steps in Machine Learning and Predictive Modelling. The data which I have used on this project work is obtained the Google Stock price data from Kaggle, which ranges from the year 2012-2016.

The profit or loss calculation is usually determined by the closing price of a stock for the day, hence consider the closing price as the target variable.

Google_Stock_Price_Train.csv (62 KB)

Detail Compact Column

Date	Open	High	Low	Close	Volume
1/3/2012	325.25	332.83	324.97	663.59	7,380,500
1/4/2012	331.27	333.87	329.08	666.45	5,749,400
1/5/2012	329.83	330.75	326.89	657.21	6,590,300
1/6/2012	328.34	328.77	323.68	648.24	5,405,900
1/9/2012	322.04	322.29	309.46	620.76	11,688,800
1/10/2012	313.7	315.72	307.3	621.43	8,824,000
1/11/2012	310.59	313.52	309.4	624.25	4,817,800
1/12/2012	314.43	315.26	312.08	627.92	3,764,400
1/13/2012	311.96	312.3	309.37	623.28	4,631,800
1/17/2012	314.81	314.81	311.67	626.86	3,832,800
1/18/2012	312.14	315.82	309.9	631.18	5,544,000
1/19/2012	319.3	319.3	314.55	637.82	12,657,800
1/20/2012	294.16	294.4	289.76	584.39	21,231,800
1/23/2012	291.91	293.23	290.49	583.92	6,851,300
1/24/2012	292.07	292.74	287.92	579.34	6,134,400

Dataset : https://www.kaggle.com/medharawat/google-stock-price?select=Google_Stock_Price_Train.csv

The training dataset in divided into two parts i.e., x-train and y-train. Each row of x-train consists of previous 60 days' independent time steps. Whereas, y-train is the output, which are called labels consists of 61st closed value, depending on x-train's previous 60 time steps. For normalizing the data, **MinMaxScaler** function from **sklearn** is used. Considering the close value, create a data structure with 60 time steps and 1 output.

ML Algorithms and Their Parameter Settings

1. **LSTM (Long Short Term Memory)**

LSTM short for Long Term Short Memory, is a type of recurrent network used for sequence prediction and have demonstrated exceptionally effective. The reason behind LSTM being very effective is because it is able to store past information that is relevant and forget the information that are not. That is why it is more capable to handle log sequences of input when compared to other recurrent neural networks that are only able to memorize short sequences. The input should always be a 3-dimensional array, where the first dimension represent the batch size, the second dimension is the time-step and the third represents the number of units in input sequence.

LSTM has three gates:

- **The input gate:** The input gate adds information to the cell state
- **The forget gate:** It removes the information that is no longer required by the model
- **The output gate:** Output Gate at LSTM selects the information to be shown as output

Modules:

Sequential - initializing the neural network
Dense - adding a densely connected neural network layer
LSTM - adding the Long Short-Term Memory layer

```
# importing the libraries
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from keras.layers import Dropout
```

Parameters:

Adam optimization is used which replaces the algorithm for gradient descent for training ML models. The default learning rate for this is 0.001 and the activation function is linear.

Hidden layers: 3 hidden layer.

return_sequences = True;

decides whether to return the full sequence or just the last sequence in the output sequence

input_shape:

shape of the training set.

```
LSTM ×
/usr/local/bin/python3.7 /Users/sanjuktadas/Pycharm_Python_Workspace/CS545-Project/LSTM.py
Using TensorFlow backend.
training_data.shape :  (1258, 1)
X_train.shape :  (1228, 30, 1)
Y_train.shape :  (1228,)
```

dropout_layers:

dropout is used in the hidden layers to avoid overfitting. specified 0.2, meaning that 20% of the layers will be dropped.

```
# Adding four LSTM layers and Dropout regularisation
# Layer-1
model.add(LSTM(units=40, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.2))

# Layer-2
model.add(LSTM(units=40, return_sequences=True))
model.add(Dropout(0.2))

# Layer-3
model.add(LSTM(units=40, return_sequences=True))
model.add(Dropout(0.2))

# Layer-4
model.add(LSTM(units=40))
model.add(Dropout(0.2))

# Adding the output layer
model.add(Dense(units=1))
```

loss function: mean_squarred_error

learning_rate = 0.001,

beta_1 =0.9,

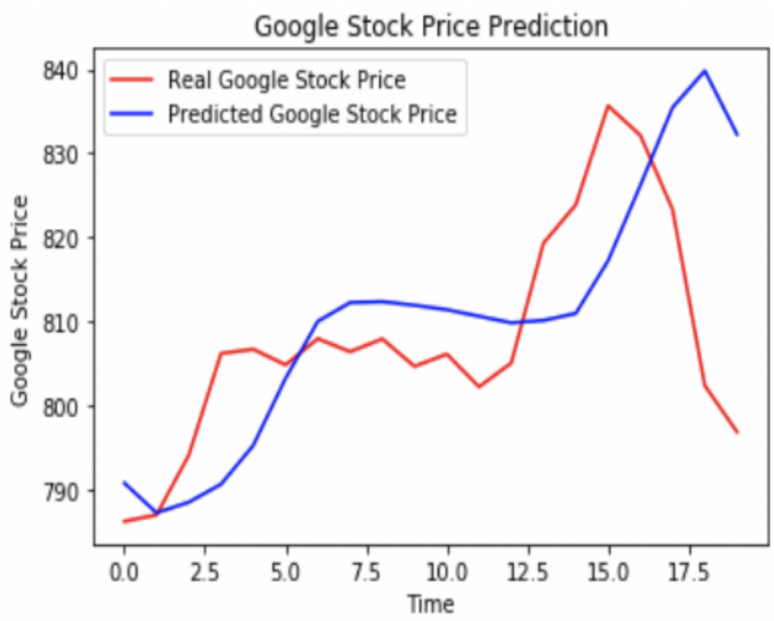
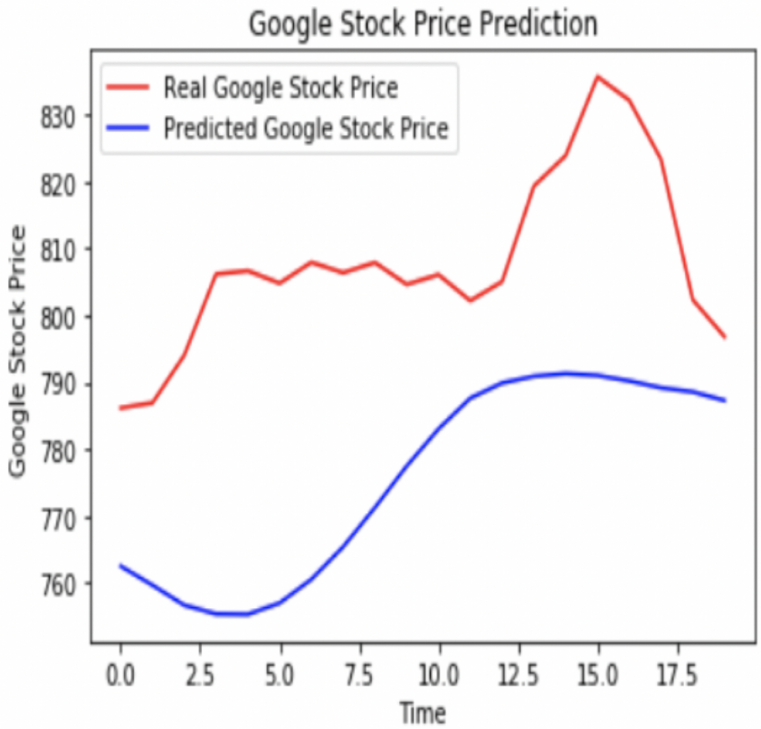
beta_2 =0.999 (beta_1=The exponential decay rate for the first moment estimates)

epochs: 100

Implementation:

In order to predict future stock prices after loading in the test set, merge the training set and the test set on the 0 axis and set the time step as 60 as done in training data set. **MinMaxScaler** is used to transform and reshape the dataset. After making the predictions I used **inverse_transform( )** to get back the stock prices in normal readable format.

Evaluation:

LSTM Prediction (60 timesteps): Mean Absolute Error: 0.0250	
LSTM Prediction (10 timesteps): Mean Absolute Error: 0.0308	

2. **SVR (Support Vector Regression)**

   In this technique, to calculate the stock prices, the overlapping segmental analysis over the given historical data is performed using the Self Organizing Maps and Support Vector Machine's derivative Support Vector Regression. A Support Vector Regression (SVR) is a type of supervised learning

algorithm that analyzes data for regression analysis unlike Support Vector Machine is used for classification.

Parameters:

rbf (radial basis function):

is used, which is a gaussian Kernel function.

C is the regularization parameter that controls the trade-off between the slack variable penalty (misclassifications) and width of the margin. Small C makes the constraints easy to ignore which leads to a large margin.

The gamma parameter defines how far the influence of a single training example reaches,

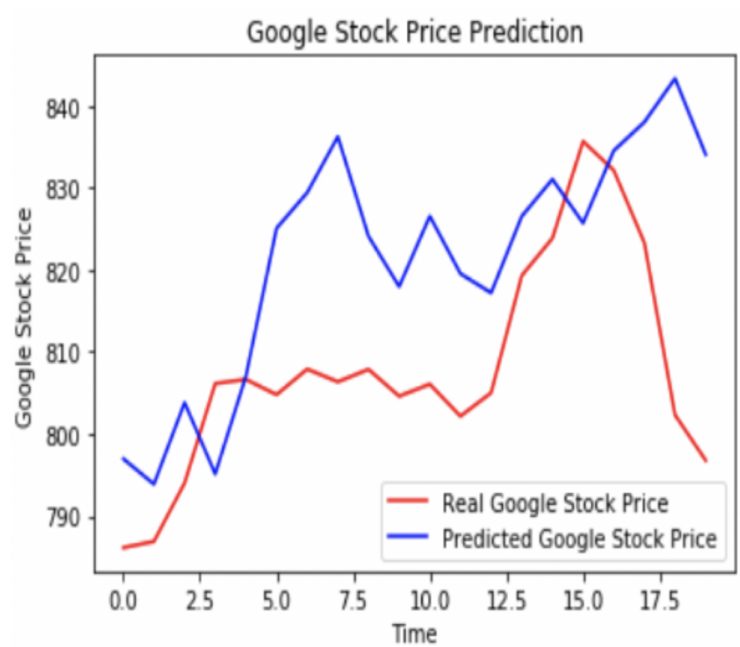
low values => 'far'

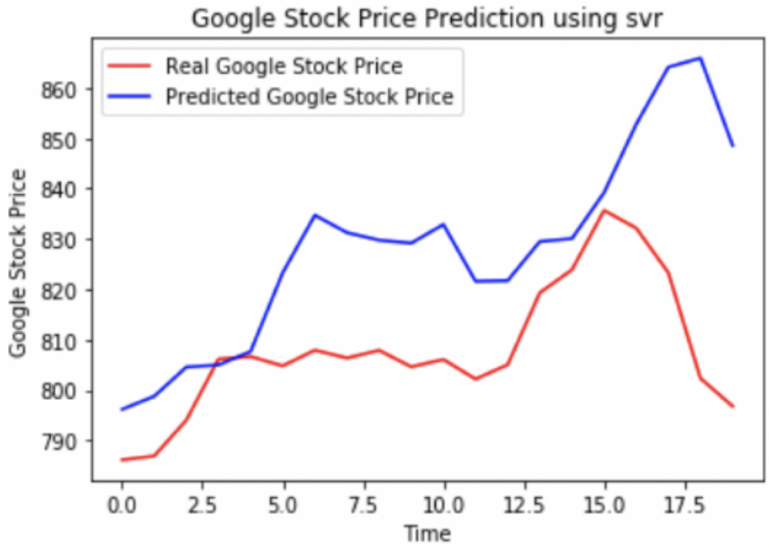
high values => 'close'.

Implementation:

Support Vector Regression model is implemented using **rbf** Kernel, the training dataset is fit into it and predict the model. In the context of price prediction, the goal is not necessarily classification into groups but estimation of real values. Therefore SVR method is used in this project, which is employed to obtain a regression model used for predicting the future stock prices.

Evaluation:

SVR Prediction (60 timesteps): Mean Absolute Error: 15.50681 -	

SVR Prediction (10 timesteps): Mean Absolute Error: 20.546351	

3. **<u>Linear Regression</u>**

Linear regression is a method used to model a relationship between a dependent variable (y), and an independent variable (x). With simple linear regression, there will only be one independent variable x. There can be many independent variables which would fall under the category of multiple linear regression. The goal of multiple regression is to model the linear relationship between your independent variables and your dependent variable.

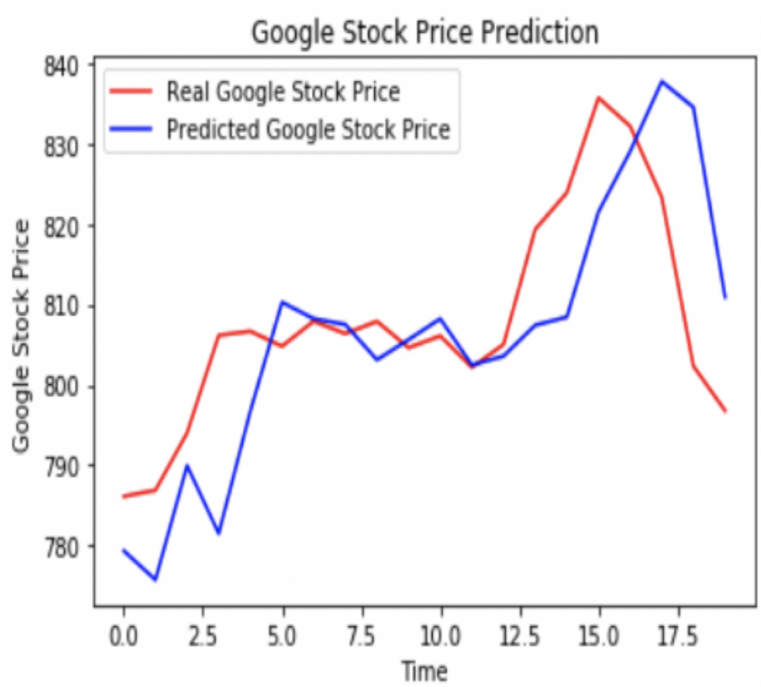
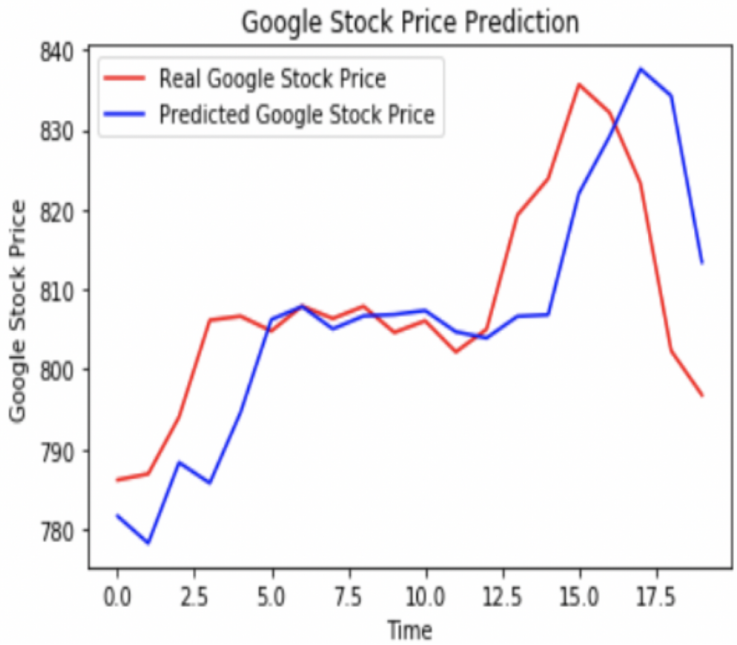
Parameters:

Since Linear Regression is one of two classifiers which have closed form solution, its object uses Ordinary Least Squares solver from **scipy**.

<u>Implementation:</u>

In statistics, linear regression is an approach for modeling the relationship between a scalar dependent variable y and one or more explanatory variables (or independent variables) denoted X. So, in the implementation, we have 60 explanatory variables, and one(y) is the dependent variable, as its value is dependent on x. It will be using scikit-learn, csv, numpy and matplotlib packages to implement and visualize linear regression. The fit method fits the X_train and y_train (x's and y's) to generate coefficient and constant for regression. Finally, the predict method finds the price(y) for the given input and returns the predicted price.

Evaluation:

LR Prediction (60 timesteps): Mean Absolute Error: 8.9111132	
LR Prediction (10 timesteps): Mean Absolute Error: 8.5977145	

Mean Absolute Error:

Mean Absolute Error (MAE) is one of the many metrics for summarizing and assessing the quality of a machine learning model. It measures the average magnitude of the errors in a set of predictions, without considering their direction. It's the average over the test sample of the absolute differences between prediction and actual observation where all individual differences have equal weight.

From graphs LSTM and LR are able to learn the model and predict the stock price where as in SVR there is no exact correlation between the real and predicted stock price. The mean absolute error of LSTM is very less compared to that of LR and SVR. This clearly shows how powerful LSTMs are for analyzing time series and sequential data.

By considering the above factors, it can be said that LSTM performed better than LR and SVR on this problem.

Conclusion:

Since this project was performed taking a comparative approach, my conclusions was significant to select the best algorithm for stock market trading in different markets. The problem is that we do not yet know ahead of time which stocks the model will be able to predict accurately and which it will not, so profiting off the model is still difficult without more experimentation.

Resources:

http://ijarcet.org/wp-content/uploads/IJARCET-VOL-6-ISSUE-9-1344-1350.pdf
https://www.researchgate.net/publication/318329563_Stock_market's_price_movement_prediction_with_LSTM_neural_networks
https://www.cs.princeton.edu/sites/default/files/uploads/saahil_madge.pdf
https://medium.com/@ewuramaminka/mean-absolute-error-mae-machine-learning-ml-b9b4afc63077
https://courses.lumenlearning.com/boundless-statistics/chapter/the-regression-line/